x86 Architecture

William Bland

September 15, 2025

Contents

1 Introduction **3**
1.1 What is the x86 architecture? . . . 3
1.2 Basics of x86 assembly . . . 3
1.3 Intel vs AT&T syntax . . . 3

2 x86 architecture **3**
2.1 Branch prediction . . . 3
2.2 Operating modes . . . 3

3 Memory **4**
3.1 Memory sizes and representaion . . . 4
3.2 Registers . . . 4
3.3 Flags . . . 7
3.4 Stack . . . 7
3.5 Heap . . . 7
3.6 Code . . . 7
3.7 Data . . . 7
3.8 Cache . . . 7
3.8.1 Cache coherency protocol . . . 7
3.8.2 Cache mappings . . . 8
3.8.3 Cache misses . . . 9
3.9 Segmentation . . . 9
3.9.1 Real Mode . . . 9
3.9.2 Protected Mode . . . 9
3.9.3 Long Mode . . . 10
3.9.4 Segment Selectors . . . 10
3.9.5 Segment descriptors . . . 10
3.9.6 GDT - Global Descriptor Table . . . 12
3.9.7 segments, sections, tss, syscalls, far jumps . . . 12
3.10 Paging . . . 12
3.10.1 Stuff . . . 12
3.10.2 Enabling Paging . . . 12
3.10.3 Paging Structure . . . 12
3.10.4 Next section lol . . . 12
3.11 Memory addressing . . . 13

4 Instructions **13**

5 Sections **16**
5.1 Data section . . . 16

6 Interrupts **16**
6.1 IDT . . . 16
6.2 ISR . . . 16
6.3 BIOS interrupts . . . 16
6.4 UEFI interrupts . . . 16
6.5 Syscalls (on Linux) . . . 16

7 Appendix **18**
7.1 Bash scripts 18
7.2 Hello world (Linux) 19
7.3 Difference (Linux) 19
7.4 Cat (Linux) 21
7.5 Hello world (Baremetal) 23

1 Introduction

1.1 What is the x86 architecture?

Processors run by executing binary instructions called machine code, this machine code can be made into a human readable format called assembly which can be turned back into machine code using a program called an assembler. Each processor has an architecture, this architecture describes how the processor executes the machine code (so different architectures execute code differently, so have different machine code and assembly languages). For multiple processors to be able to execute the same machine code they must have the same architecture, some popular architectures include ARM, PowerPC, RISC-V and x86. x86 is an example of a Complex Instruction Set Computer (CISC), this means that there are lots of instructions so a lot can be done in a single instruction so is very fast, this however uses a lot of power so is the main architecture in desktop computers. This is in opposition to ARM which is a Reduced Instruction Set Computer (RISC) which has less instructions so has to execute many to perform the same task so is slower than x86, this however uses less power so is the main architecture in mobile devices and embedded systems.

1.2 Basics of x86 assembly

x86 assembly code contains one instruction per line, the line starts with the name of the instruction followed by its parameters which can either be registers, a memory address or an immediate value (a constant). Lines and constants can be given labels and can be referred to in instructions. The assembler only assembles one file at a time so if a label (symbol) is located in another file, the assembler will fail to assemble the program; this is fixed in NASM by marking the symbol in the file its defined in as global (so its accessible to the linker) and as extern in the file its used in. A program called the linker will combine these object files into a single executable and resolve all extern symbols to their global definitions. Furthermore the linker makes the executable in the correct format for the target platform (eg elf64 on Linux). Lastly, all code written will be using the Intel syntax using the NASM assembler and will be linked using the ld linker unless stated otherwise.

1.3 Intel vs AT&T syntax

x86 assembly has two different syntaxes: Intel and AT&T.

???	**Intel**	**AT&T**
a	b	c

2 x86 architecture

2.1 Branch prediction

Spectre/meltdown

2.2 Operating modes

x86 processors have several different operating modes that they can run in, these operating modes can change the addressing mode, available instruction set and registers. x86 processors start up in real mode from address 0xFFFF0.

Name	Description
Real mode	Also called real address mode, all addresses always correspond to physical (real) memory addresses. Memory addresses are 20 bit allowing for 1MiB of memory, they are referenced by the value in a segment register left shifted by 4 then added to an offset for example [es:di]. There is no memory protection in real mode, and all x86 CPU's start in real mode when reset for compatibility
Unreal mode	Also called big real mode, it is a variant of real mode, it allows the access to 4GiB of memory
Protected mode	Also called protected virtual address mode, it allows the use of segmentation, virtual memory, paging and safe multi-tasking. To enter protected mode the software must setup a descriptor table and enable the Protection Enable (PE) bit in CR0 (Intel 80286).
Virtual 8086 mode	Also called virtual read mode, it allows real mode processes to run in protected mode. It is a hardware virtualisation technique and uses the 20-bit segmentation that real mode uses but is subject to the protected mode's memory paging.
Long mode	The 64-bit OS can access 64-bit registers and instructions. 64-bit programs run in a sub-mode called 64-bit mode, and 32-bit and 16-bit protected mode programs run in another sub-mode called compatibility mode, however real mode and virtual 8086 mode programs cannot run natively in long mode.
x86 virtualisation mode	Mode that uses hardware assisted virtualisation.
AIS and 8080 emulation mode	Rarely used operating modes found on only a few old CPUs.

3 Memory

3.1 Memory sizes and representaion

Processors operate on digital signals which can either be high or low and can be interpreted in binary as a 1 or a 0, a single 1 or 0 is a bit.

Table 1: Data sizes

Name	Unit	Bits	Bytes
bit	b	1	1/8
nibble	-	4	1/2
byte	B	8	1
word	-	16	2
dword (double word)	-	32	4
qword (quad word)	-	64	8

Lastly, all memory addresses point to a unique byte in memory therefore a multibyte integer (eg 64-bit) could be stored in two different orderings (its endianness). In Little-Endian (LE) ordering the least significant byte occupies the smaller address space, while in Big-Endian (BE) ordering the most significant byte occupies the smaller address space. x86 and most other computers uses LE while data transmitted over networks uses BE.

3.2 Registers

Resisters store small values in small regions of memory directly accessible to the current CPU thread. The registers on the same line overlap in memory (eg the 32-bit register is located in the lower half of the 64-bit register)

Table 2: General purpose registers

64-bit	32-bit	16-bit	Higher 8-bit	Lower 8-bit	Description
RAX	EAX	AX	AH	AL	Accumulator
RBX	EBX	BX	BH	BL	Base
RCX	ECX	CX	CH	CL	Counter
RDX	EDX	DX	DH	DL	Data (often extends the accumulator)
RSI	ESI	SI	-	SIL	Source index for string operations
RDI	EDI	DI	-	DIL	Destination index for string operations
RSP	ESP	SP	-	SPL	Stack pointer
RBP	EBP	BP	-	BPL	Base pointer (for stack frames)
R8	R8D	R8W	-	R8B	General purpose register
R9	R9D	R9W	-	R9B	General purpose register
R10	R10D	R10W	-	R10B	General purpose register
R11	R11D	R11W	-	R11B	General purpose register
R12	R12D	R12W	-	R12B	General purpose register
R13	R13D	R13W	-	R13B	General purpose register
R14	R14D	R14W	-	R14B	General purpose register
R15	R15D	R15W	-	R15B	General purpose register

Registers 8-15, and the SIL, DIL, SPL and BPL are only available in 64-bit programs since they were added in the 64-bit extensions There are also segment registers, these are used in memory segmentation (See Segmentation 3.9). The segment registers (always 16-bit) are:

Register	Description
CS	Code segment
DS	Data segment
SS	Stack segment
ES	Extra segment (for string operations)
FS	General purpose segment register (Since Intel 80386)
GS	General purpose segment register (Since Intel 80386)

There are also several control registers, they contain flags that control the operation of the CPU. Control registers not listed in the table below such as CR1 and bits not listed are reserved and should be 0 (writing to a reserved register results in a #UD exception, setting a reserved bit in CR0[0:31] is ignored, setting a reserved bit in CR0[32:63], CR4 or CR3[MAXPHYADDR:63] causes a #GP). When protection is enabled, control registers can only be written to at CPL 0, however CR2 is writable by software. On a 64-bit processor, writing to a control register outside of 64-bit mode sets the upper 32 bits to 0, however when transitioning out of IA-32e mode, the registers retain their upper 32 bits. CR8 is only available in 64-bit mode, and CR4[32:63] only affects long mode and can only be changed in long mode. Below is a list of the control registers and their flags:

Bits	Label	Description
		CR0
Contains system control flags that control operating mode and states of the processor		
0	PE	Protected Mode Enable
1	MP	Monitor co-processor
2	EM	x87 FPU Emulation
3	TS	Task switched
4	ET	Extension type
5	NE	Numeric error
16	WP	Write protect
18	AM	Alignment mask
29	NW	Non-write through

30	CD	Cache disable
31	PG	Paging
CR2		
Contains linear address that caused a page fault		
0:31(63)	PFLA	Page Fault Linear Address
CR3		
Contains the physical address of the base of the paging-structure hierarchy and two flags		
3	PWT	Page-level Write-Through (not used in PAE or if bit CR4.PCIDE = 1 in long mode)
4	PCD	Page-level Cache Disable (not used in PAE or if bit CR4.PCIDE = 1 in long mode)
12:31(63)	PDBR	Page Directory Base Register (PAE = Base of PDPT, long = base of PML4T/PML5T)
CR4		
Contains flags that enable several architectural extentions		
0	VME	Virtual 8086 Mode Extensions
1	PVI	Protected-mode Virtual Interrupts
2	TSD	Time Stamp Disable
3	DE	Debugging Extensions
4	PSE	Page Size Estension
5	PAE	Physical Address Extension
6	MCE	Machine Check Extension
7	PGE	Page Global Enabled
8	PCE	Performance-Monitoring Counter enable
9	OSFXSR	Operating System Support for FXSAVE and FXRSTOR Instructions
10	OSXMMEXCPT	Operating System Support for Unmasked SIMD Floating-Point Exceptions
11	UMIP	User-Mode Instruction Prevention (if set, #GP on SGDT, SIDT, SLDT, SMSW, and STR instructions when CPL > 0)
12	LA57	57-bit linear addresses (if set, the processor uses 5-level paging otherwise it uses 4-level paging)
13	VMXE	Virtual Machine Extensions Enable
14	SMXE	Safer Mode Extensions enable
16	FSGSBASE	Enables the instructions: RDFSBASE, RDGSBASE, WRFSBASE and WRGSBASE
17	PCIDE	PCID Enable
18	OSXSAVE	XSAVE and Processor Extended States Enable
20	SMEP	Supervisor Mode Execution Protection Enable
21	SMAP	Supervisor Mode Access Prevention Enable
22	PKE	Protection Key Enable
23	CET	Control-flow Enforcement Technology
24	PKS	Enable Protection Keys for Supervisor-Mode Pages
CR8		
Provides access to the Task Priority Register (TPR)		
0:3	TPL	Task Priority Level

3.3 Flags

3.4 Stack

3.5 Heap

3.6 Code

3.7 Data

3.8 Cache

Cache is a relatively small amount of memory placed within the processor. It is extremely fast SRAM (as opposed to the slower main memory DRAM). There are typically several layers of cache, with each layer getting larger in capacity and slower to access:

1. L1d:
   Extremely fast data cache
2. L1i:
   Extremely fast instruction cache
3. L2:
   Very fast cache
4. L3:
   Fast cache (shared between cores)

3.8.1 Cache coherency protocol

Cache coherency protocols ensure the caches on difference processors [cores] store the most recent, correct copy of data. x86 uses MSI-based protocols, such as MESI, MOESI and MESIF (with MESI being used in older processors, and the latter two being used in newer AMD and Intel processors respectively); MSI-based protocols are a type of write-invalidate snooping protocol, this means that a write on one cache invalidates all other cache. Each cache line is stored with a state, when a cacheline is accessed (read miss or write), its state is updated and it sends a message to all other processors on a bus. The cache controllers snoops the bus, if a message involves a cacheline in its cache, its state is updated to reflect the message.
The accesses to cache which affect state are:

1. PR - current Processor Reads (cache miss)
2. PW - current Processor Writes
3. BR - Bus Read (cache miss, another processor reads)
4. BW - Bus Write (another processor writes)

The state transitions are defined by a finite state machine, the invalid state is considered as a cache miss and uncached items can be considered to be in the invalid state.

State	Data	Responds to BR	Description
Modified	Dirty	Yes	Only copy of the data
Exclusive	Clean	Yes	Only copy of the data
Shared	Clean*	No**	Multiple copies of the data
Invalid	Invalid	No	The cache line does not hold a valid copy of the data
Owned	Dirty	Yes	Multiple copies of the data (all others are shared)
Forward	Clean	Yes	Multiple copies of the data (all others are shared)

* Dirty if another cache is in the Owned state.
** Will never respond (provide the cached data) if another cache is Owned or Forward, otherwise depending on implementation either a single Shared cache will respond, they all respond or none respond (most likely).
Owned and Modified states write to main memory (or next-level cache) when switching to a clean state or when evicted from cache. Furthermore, states with the only copy of cache (Modified and Exclusive) do not have to send a bus message when writing to its cache. The following finite state machines represent the transitions between the states of their respective protocols (MSI has been included since it is the foundation of the protocols but is not used in x86).

	Access			
State	**PR**	**PW**	**BR**	**BW**
MSI transitions				
M	M	M	S	I
S	S	M	S	I
I	S	M	I	I
MESI transitions				
M	M	M	S	I
E	E	M	S	I
S	S	M	S	I
I	*	M	I	I
MOESI transitions				
M	M	M	O	I
O	O	M	O	I
E	E	M	S	I
S	S	M	S	I
I	*	M	I	I
MESIF transitions				
M	M	M	S	I
E	E	M	S	I
S	S	M	S	I
I	**	M	I	I
F	F	M	S	I

* Transitions to E if exclusive or S if shared.
** Transitions to E if exclusive or F if shared.

3.8.2 Cache mappings

Cache mapping describes how the physical address is used to map to the data, there are three types:

1. Direct:
   Each physical address is split into three parts: tag, block and word. The block is used to identify a specific cacheline, each cacheline stores a tag and the tags are compared, if equal its considered a cache hit else it is a cache miss. On a hit, the word is used as an offset into the cacheline to retrieve the data. On a miss, the new data is fetched from main memory and replaces the cacheline.

2. Associative:
   Each physical address is split into two parts: tag and word. Every cacheline is checked in parallel for if the tags are equal. On a hit, the word is used as an offset into the cacheline to retrieve the data. On a miss, the new data is fetched from main memory and replaces a cacheline usually identified by a LRU algorithm (Least Recently Used).

3. N-Way Set-Associative:
   Each physical address is split into three parts: tag, block and word. The block is used to identify a specific set (each set stores N cachelines). Each cacheline in the set is checked in parallel for if the tags are equal. On a hit, the word is used as an offset into the cacheline to retrieve the data. On a miss, the new data is fetched from main memory and replaces a cacheline within the set usually identified by a LRU algorithm.

Mapping	**Pros**	**Cons**
Direct	Fast (spacial locality)	Cache lines often get evicted if they have the same block
Associative	Cache lines only get evicted if not used (temporal locality)	Slow
Set-Associative	Faster than Associative with similar positives	Slower than Direct and Conflict misses can occur

Most x86 processors use N-Way Set-Associative caches, with the more performant ones using larger set sizes.

3.8.3 Cache misses

A cache miss occurs when valid data does not exist in the processor's cache. The first three types of cache miss are commonly known as the three C's.

1. Compulsory (cold) miss:
   Occurs when accessing memory for the first time (so was never cached)

2. Capacity miss:
   Occurs when cache is full and a cacheline was evicted but later used

3. Conflict miss:
   Occurs when multiple cache lines are mapped to the same location in cache, causing one to get evicted so if read again later it is a miss

4. Coherence miss:
   The (write-invalidate) cache coherency protocol (eg MESI) invalidated the cacheline because it was written in another processor

3.9 Segmentation

Segmentation is a method of memory management that separates parts of a processes address space into different segments (such as code and data), it uses the 6 segment registers to keep track of the segments. It is used to convert a logical address (which consists of a segment register and an offset) into a linear address. Segmentation has different behaviours depending on what operating mode the processor is in, however unlike paging it is always enabled.

3.9.1 Real Mode

The value in the segment register is left shifted by 4 (multiplied by 16) then added to the offset to produce a 20-bit physical address allowing up to 1MiB of memory instead of the 16-bit limit of 64KiB. The kernel allocates memory for each running process and gives them each different segments, this allows the programs to be able to be loaded into any part of the address space and to not affect other program's memory. However there is no hardware enforcement of memory protection as any process can read and modify any memory location.

3.9.2 Protected Mode

The value in the segment register is a segment selector, it is used as an index into the LDT/GDT to select a segment descriptor. Each segment descriptor contains a base address, limit and flags. The base address is a linear address and points to the base of the segment, the limit is the size of the segment and the flags set the segments properties such as permissions, 32 or 16 bit code and granularity of the limit. There are many models which can be used to implement segmentation; some common models are listed below, other models exist and additional functionality can be added to the models such using the code segment to point to ROM.

1. Basic Flat Model:
   Contains at least two segment descriptors, one for code and one for data (this is because the code descriptor has to be executable so is usually read-only). Both descriptors are mapped to the entire linear address space (even if the physical memory does not exist) so it has a base address of 0 and a limit of 4GiB. This essentially disables segmentation for both the application and system developer, and since the entire linear address space is mapped no #GP exceptions will occur. Usually implemented alongside paging.

2. Protected Flat Model:
   Similar to the Basic Flat Model but only included the range of addresses for which physical memory actually exists, so the limit is set to the capacity of physical memory. If a process accesses a non-existent memory location a #GP exception will occur.

3. Multi-Segment Model:
   Uses the full capabilities of segmentation to provide hardware enforced protection of memory. Each program is given its own segment descriptor table and multiple segments (which may be private or shared with other programs). Hardware checks can be used to enforce the program to access memory it owns and in a way the OS allows (such as not writing to code/read-only memory)

3.9.3 Long Mode

The behaviour of segmentation in long mode depends if it is running in 64-bit or compatibility mode. In compatibility mode it functions the same as it does in protected mode. In 64-bit mode, segmentation is generally disabled (but not completely), which creates a flat 64-bit linear address space. The base address of the CS, DS, ES and SS are treated as 0 (the effective address is equal to the linear address), and the processor does not perform limit checks (on all segment registers). The FS and GS registers are often used to reference thread-local or kernel data structures since their base address still applies.

3.9.4 Segment Selectors

Segment selectors are used as a 16-bit identifier for a segment descriptor. Segment selectors contain 3 parts:

1. Bits 3:15 - Index:
   The index into the descriptor table, selects one of the 8192 segment descriptors in the table. Its value is multiplied by 8 then added to the base address of the of the descriptor table.

2. Bit 2 - Table Indicator (TI) flag:
   Selects between the two descriptor tables: 0 selects the GDT to be used, 1 selects the LDT to be used.

3. Bits 0:1 - Requested Privilege Level (RPL):
   Specifies the privilege level of the selector.

3.9.5 Segment descriptors

Segment descriptors are identifiers for segments of memory. They contain multiple parts which can be categorised into: (linear) base address of the segment, limit (size of segment - 1), flags (controls the abilities of the segment).

Bits	**Size**	**Name**	**Use**
63:56	8	Base 31:24	1 byte of the base address
55:55	1	Granularity (G)	0 = limit in bytes, 1 = limit in 4-KiB
54:54	1	D/B	Default operation size, 0 = 16-bit, 1 = 32-bit
53:53	1	L	1 = 64-bit code segment (IA-32e mode only)
52:52	1	AVL	Available for use by system software
51:48	4	Limit 19:16	4 bits of the limit
47:47	1	Present (P)	0 = segment not present, 1 = segment present
46:45	2	DPL	Descriptor privilege level
44:44	1	S	Descriptor type, 0 = system, 1 = code/data
43:40	4	Type	Segment type
39:16	24	Base 23:00	3 byte of the base address
00:16	16	Limit 15:00	2 bytes of the base address

The segment types are described below:

	Type					
S	**3**	**2**	**1**	**0**	**32-bit mode**	**IA-32e mode**
0	0	0	0	0	Reserved	Reserved
0	0	0	0	1	16-bit TSS (Available)	Reserved
0	0	0	1	0	LDT	LDT
0	0	0	1	1	16-bit TSS (Busy)	Reserved
0	0	1	0	0	16-bit Call Gate	Reserved
0	0	1	0	1	Task Gate	Reserved
0	0	1	1	0	16-bit Interrupt Gate	Reserved
0	0	1	1	1	16-bit Trap Gate	Reserved
0	1	0	0	0	Reserved	Reserved
0	1	0	0	1	32-bit TSS (Available)	64-bit TSS (Available)
0	1	0	1	0	Reserved	Reserved
0	1	0	1	1	32-bit TSS (Busy)	64-bit TSS (Busy)
0	1	1	0	0	32-bit Call Gate	64-bit Call Gate
0	1	1	0	1	Reserved	Reserved
0	1	1	1	0	32-bit Interrupt Gate	64-bit Interrupt Gate
0	1	1	1	1	32-bit Trap Gate	64-bit Trap Gate
-	**-**	**E**	**W**	**A**	**Type**	**Description**
1	0	0	0	0	Data	Read-Only
1	0	0	0	1	Data	Read-Only, accessed
1	0	0	1	0	Data	Read-Write
1	0	0	1	1	Data	Read-Write, accessed
1	0	1	0	0	Data	Read-Only, expand-down
1	0	1	0	1	Data	Read-Only, expand-down, accessed
1	0	1	1	0	Data	Read-Write, expand-down
1	0	1	1	1	Data	Read-Write, expand-down, accessed
-	**-**	**C**	**R**	**A**	**Type**	**Description**
1	1	0	0	0	Code	Execute-Only
1	1	0	0	1	Code	Execute-Only, accessed
1	1	0	1	0	Code	Execute-Read
1	1	0	1	1	Code	Execute-Read, accessed
1	1	1	0	0	Code	Execute-Only, conforming
1	1	1	0	1	Code	Execute-Only, conforming, accessed
1	1	1	1	0	Code	Execute-Read, conforming
1	1	1	1	1	Code	Execute-Read, conforming, accessed

Many of the bits are described below:

1. E = Expansion-direction

2. W = Write-enabled:
   Allows the segment to be written; the stack segment must be writeable, loading the SS register with a read only segment causes a #GP exception.

3. A = Accessed:
   The processor sets it to 1 if it accessed the segment. The kernel can clear its value and it is often useful for debugging. If the segment descriptor is placed in ROM, the processor may enter an infinite loop attempting to set the Accessed bit. To prevent this, the segment descriptors in ROM should have their Accessed bit set.

4. C = Conforming:
   A transfer of execution into a more privileged conforming segment will continue at the current privilege level. Transferring execution into a more privileged non-conforming segment or less privileged segment will result in a #GP exception if a call or task gate is not used. All data segments are non-conforming so cannot be accessed

by less privileged processes, however unlike code segments higher privileged processes can access lower privileged data segments without the use of a gate.

5. R = Read-enabled:
   Allows the code segment to be read and is useful if constants or static data are placed within the code segment. The code segments can be read either by using a CS override or by loading the segment into a data-segment register (DS, ES, GS, FS).

3.9.6 GDT - Global Descriptor Table

Contains the segment descriptors, its base address (linear) and limit (number of bytes in the GDT - 1) are stored in the GDTR (GDT Register). The `LGDT` instruction stores the base address and limit from a memory location into the GDTR. The 0th descriptor of the GDT is often used to store the address and limit since it will never be loaded as a descriptor since the selector would be null.

3.9.7 segments, sections, tss, syscalls, far jumps

3.10 Paging

Paging is a method of memory management that separates a programs address space (linear address) from the physical address space. It occurs after segmentation but unlike segmentation it does not need to be enabled, if it is disabled linear addresses are the same as physical addresses. The OS splits up the physical address space into 4KiB pages, each process has a paging structure which the OS manages, it consists of many layers; each layer consists of the entries for either the physical address of the next layer or page and flags. When a process accesses memory, the processor hardware converts the linear address into its corresponding physical address using the paging structure (by traversing the layers) and then uses it to access memory, this is called a hardware-assisted page walk.

3.10.1 Stuff

CR3 contains the physical address of the first paging structure unless HLAT (Hypervisor-managed Linear-Address-Translation) paging is used where it is stored in HLATP.

3.10.2 Enabling Paging

Before paging is enabled, CR3 should contain the physical address of the first paging structure, and protection (CR0.PE) is enabled. Furthermore, once paging is enabled a different paging method can be used but paging should be disabled first before changing flags since exceptions may occur by changing the flags whilst paging is enabled. The table below shows which combination of flags is used to enable which type of paging, once the flags are set, set CR0.PG to enable paging.

Method	CR4.PAE	IA32_EFER.LME	CR4.LA57
32-bit	0	0	-
PAE	1	0	-
4-level	1	1	0
5-level	1	1	1

Since IA32_EFER.LME enables long mode (IA-32e), 32-bit and PAE can only be used in legacy protected mode and 4-level and 5-level can only be used in long mode. Furthermore, in compatibility mode (sub mode of IA-32e), bits 63-32 of the linear address are treated as 0.

3.10.3 Paging Structure

3.10.4 Next section lol

5.2 - Paging structure
Each structure is 4096 bytes in size and comprises a number of individual entries. In 32-bit paging each entry is 32 bits (4 bytes) so there are 1024 entries in each structure. In 64-bit paging each entry is 64 bits (8 bytes) there are thus 512 entries in each structure. (In PAE however there is a structure 32 bytes in size containing 4 64-bit entries)
The processor uses the upper portion of a linear address to identify a series of paging-structure entries. The last of these entries identifies the physical address of the region to which the linear address translates (called the page frame). The lower portion of the linear address (called the page offset) identifies the specific address within that region to which the linear address translates.

Each paging structure entry contains a physical address - either the address of another paging structure or a page frame. In the first case it is said to reference the other paging structure; in the second the entry is said to map a page. CR3 contains the first paging structure. (unless HLAT paging)
A paging-structure entry is marked as not present if its P flag (bit 0) is clear, if it is encountered during translation it causes a page-fault exception.
Usually an address is mapped to a 4 KiB page, if it is not the final paging structure then bit 7 (PS - page size) is checked, if it is 0 the process continues normally, else if it is 1 the entry maps a larger page (4MiB for 32-bit paging if CR4.PSE = 1, 2MiB for other paging modes regardless of CR4.PSE; 4-level and 5-level paging may also support 1GiB pages). Bit 7 is used for other purposes if its in the final paging structure.
paging structure:

Paging Structure	**Entry Name**	**Paging Mode**	**Physical Address of Structure**	**Bits Selecting Entry**	**Page Mapping**
PML5 table	PML5E	32-bit, PAE, 4-level	N/A		
		5-level	CR3	56:48	N/A (PS must be 0)
PML4 table	PML4E	32-bit, PAE	N/A		
		4-level	CR3	47:39	N/A (PS must be 0)
		5-level	PML5E		
Page-directory-pointer table	PDPTE	32-bit	N/A		
		PAE	CR3	31:30	N/A (PS must be 0)
		4-level, 5-level	PML4E	38:30	1-GiB page if PS=1
Page directory	PDE	32-bit	CR3	31:22	4-MiB page if PS=1
		PAE, 4-level, 5-level	PDPTE	29:21	2-MiB page if PS=1
Page table	PTE	32-bit	PDE	21:12	4-KiB page
		PAE, 4-level, 5-level		29:21	

5.3 - 32-bit paging
5.4 - PAE paging
5.5 - 4 and 5 level paging
5.6 - Access rights
5.7 - page fault exceptions
5.8-5.11 - ???
5.12 - virtual memory

3.11 Memory addressing

4 Instructions

The format of the instruction is `instruction op1, op2, op3` eg `'MOV RAX, 10'`. Each instruction has different limitations to what type an operand can be, these types are

Symbol	**Name**	**Description**
r	General purpose register	A register such as eax
m	Memory address	A memory address such as `[eax + 4*ebx + 4]`
i	Immediate	A constant value such as 80h or a pointer to a label or data section
s	Segment Register	A segment register such as es

The types of an operand or identical instructions with different names will be separated by `(|)`, the size in bits may also be appended to the type and (') may be used to exclude a single size (eg `i8|i16` means an 8 or 16 bit immediate value and `r'8` means a 16, 32 or 64 bit register). Some instructions have more than one row of operands, each row shows a different combination of operand types. Most instructions should only operate on a single word size (eg 64-bit), if an instruction is used with different sized operands then it may cause an assembler error, cause an invalid opcode exception, truncate the source or bit extend it (either arithmetically or logically). Furthermore many of these instructions set/clear flags, for example the OF (overflow flag) is set on signed overflow, CF (carry flag) is set on unsigned overflow and ZF (zero flag) is set when the result is zero. Some instructions can be prefixed with LOCK, this

makes the instruction atomic and will give the thread exclusive access to the cache line it is accessing; LOCK can only be used when the destination is a memory location and the instruction supports it else it will raise a UD exception.

Instruction	OP 1	OP 2	OP 3
Memory			
`MOV` - Move Copies the value from OP2 into OP1	r	r|m|i|s	-
	m	r|i|s	-
	s	r|m	-
`LEA` - Load Effective Address Stores the address from OP2 into OP1 but unlike MOV it does not dereference the address	r'8	m	-
`[LOCK] CMPXCHG` - Compare and Exchange Compares `AL|AX|EAX|RAX` with OP1 (dst), if equal OP2 (src) is written into OP1 and ZF is set else OP1 is written into the accumulator and ZF is cleared	r|m	r	-
`[LOCK] XCHG` - Exchange Swaps (exchanges) the values held in OP1 and OP2	r	r|m	-
	r|m	r	-
Control			
`UD` - Undefined Instruction Causes an invalid opcode exception	-	-	-
`NOP` - No Operation Performs no operation, the first encoding is a single byte NOP and is an alias for XCHG (E)AX, (E)AX, and the second encoding is a multibyte NOP and may not be supported on all processors	-	-	-
	r|m	-	-
`CMP` - Compare Two Operands Compares OP1 with OP2, it does this by subtracting OP2 from OP1, setting the relevant flag and discarding the result	r	r|m	-
	r|m	r	-
	r|m	i'64	-
`TEST` - Logical Compare Performs a bitwise logical AND on OP1 and OP2, sets the relevant flags and discards the result	r|m	r	-
	r|m	i'64	-
`Jcc` - Jump If Condition Is Met Jumps if the condition is met	i'64	-	-
Functions			
`PUSH` - Push data onto the stack Decrements the Stack Pointer by the operand size, then moves OP1 to the memory location pointed to by the it. It is recommended to only push values the same size as the pointer size to prevent a misaligned stack. Values less than the operand size are sign-extended to the operand size however segment registers are zero-extended. The operand size can be overridden by prefixes however its based on the current stack segment D flag. In long mode, 32-bit registers and the CS,DS,ES and SS registers cannot be encoded	(r|m)'8	-	-
	i'64	-	-
	s	-	-
`POP` - Pop data off the stack Moves the value pointed to by the Stack Pointer to OP1 then increments it by the operand size. Uses the same rules as PUSH	(r|m)'8	-	-
	s	-	-
Arithmetic			

`[LOCK] ADD` - Add Adds OP2 to OP1. If OP2 is an `immediate` value and its word size is less than OP1 then it is sign-extended	r	r|m	-
	r|m	r	-
	r|m	i'64	-
`[LOCK] INC` - Increment Increments the value in OP1 by 1	r|m	-	-
`[LOCK] ADC` - Add With Carry Adds (OP2 + CF) to OP1. If OP2 is an `immediate` value and its word size is less than OP1 then it is sign-extended	r	r|m	-
	r|m	r	-
	r|m	i'64	-
`[LOCK] SUB` - Subtract Subtracts OP2 from OP1. If OP2 is an `immediate` value and its word size is less than OP1 then it is sign-extended	r	r|m	-
	r|m	r	-
	r|m	i'64	-
`[LOCK] DEC` - Decrement Decrements the value in OP1 by 1	r|m	-	-
`MUL` - Unsigned Multiply Performs unsigned integer multiplication on `AL|AX|EAX|RAX` and OP1, and stores the result in `AX|DX:AX|EDX:EAX|RDX:RAX`	r|m	-	-
`IMUL` - Signed Multipy Multiplies signed integers. In the first encoding `AL|AX|EAX|RAX` is multiplied by OP1 and is stored in `AX|DX:AX|EDX:EAX|RDX:RAX`. In the second OP1 is multiplied by OP2, the result is truncated and stored in OP1. In the third OP1 is multiplied by the product of OP2 and OP3, the result is truncated and stored in OP1	r|m	-	-
	r	r|m	-
	r	r|m	i'64
`DIV` - Unsigned Divide Divides unsigned integers. It divides `AX|DX:AX|EDX:EAX|RDX:RAX` by OP1, stores the quotient in `AL|AX|EAX|RAX` and stores the remainder in `AH|DX|EDX|RDX`. Causes a Divide Error exception if OP1 is 0 or the quotient is larger than its destination register	r|m	-	-
`IDIV` - Signed Divide Divides signed integers. It divides `AX|DX:AX|EDX:EAX|RDX:RAX` by OP1, stores the quotient in `AL|AX|EAX|RAX` and stores the remainder in `AH|DX|EDX|RDX`. Causes a Divide Error exception if OP1 is 0 or the quotient is larger than its destination register	r|m	-	-
`[LOCK] NEG` - Two's Complement Negation Replaces OP1 with its two's complement (the same as subtracting it from 0)	r|m	-	-
Logical			
`BT` - Bit Test Stores the OP2 (mod OP1 size) bit in OP1 into CF	r|m	r|i8	-
`[LOCK] BTC` - Bit Test and Complement Stores the OP2 (mod OP1 size) bit in OP1 into CF and complements the bit in OP2	r|m	r|i8	-
`[LOCK] BTR` - Bit Test and Reset Stores the OP2 (mod OP1 size) bit in OP1 into CF and resets the bit (to 0) in OP2	r|m	r|i8	-
`[LOCK] BTS` - Bit Test and Set Stores the OP2 (mod OP1 size) bit in OP1 into CF and sets the bit (to 1) in OP2	r|m	r|i8	-

`[LOCK] XOR` - Logical Exclusive OR Performs a bitwise XOR on OP1 and OP2 and stores the result in OP1	`r|m`	`i'64`	-
	`r|m`	`r`	-
	`r`	`r|m`	-
`[LOCK] AND` - Logical AND Performs a bitwise AND on OP1 and OP2 and stores the result in OP1	`r|m`	`i'64`	-
	`r|m`	`r`	-
	`r`	`r|m`	-
`[LOCK] OR` - Logical Inclusive OR Performs a bitwise OR on OP1 and OP2 and stores the result in OP1	`r|m`	`i'64`	-
	`r|m`	`r`	-
	`r`	`r|m`	-

In long mode, to access many of its features the REX prefix has to be placed before an instruction. REX must be encoded to access 64-bit operands (if the instruction does not default to it) an extended register (R8-15, XMM8-15, YMM8-15, CR8-15, DR8-15) Uniform byte register (SPL, BPL, SIL, DIL) it cannot be encoded when using a high byte register (AH, BH, CH, DH) default 64-bit instructions in long mode are: CALL (near), ENTER, LEAVE, LGDT, LIDT, LLDT, LOOP, LOOPcc, LTR, MOV CR(n), MOV DR(n), POP r—m, POP r, POP FS, POP GS, POPFQ, PUSH i8, PUSH i32, PUSH r—m, PUSH r, PUSH FS, PUSH GS, PUSHFQ, RET (near) Almost every assembler automatically uses the nessissary REX prefixes when required so it is not needed to add It can be done by using db followed by the required byte in the line above the instruction Sometimes REX can cause AH,BH,CH,DH to be interpreted as SPL, BPL, SIL, DIL

5 Sections

5.1 Data section

db, equ, $ etc

6 Interrupts

Interrupts are signals from devices to the CPU which makes it immediately pause execution and run code to handle the interrupt. When an interrupt occurs, the CPU finds its entry in an OS provided lookup table called the Interrupt Descriptor Table (IDT) and can have up to 256 entries; once the CPU locates the entry, it jumps to the code to handle the interrupt which is called the Interrupt Service Routine (ISR) or interrupt handler.
There are three types of interrupts:

1. Exceptions: Generated internally by the CPU and are used to alert the kernel of an event which requires its attention such as a Double Fault, Page Fault, General Protection Fault, etc
2. Interrupt Request (IQR) or Hardware Interrupt:
3. C

6.1 IDT

6.2 ISR

6.3 BIOS interrupts

6.4 UEFI interrupts

6.5 Syscalls (on Linux)

In order for a program to communicate with the OS it has to hand over control of the processor back to the OS. In 32-bit programs it uses the interrupt `int 80h`, in 64-bit programs it uses the instruction `syscall`. The parameters for the syscall are placed in specific registers.

Table 5: Registers used for syscall parameters (not all args are used for all syscalls)

i386 Registers	x86-64 Registers	Usage
EBX	RDI	Arg1
ECX	RSI	Arg2
EDX	RDX	Arg3
ESI	R10	Arg4
EDI	R8	Arg5
EBP	R9	Arg6
EAX	RAX	Syscall number (different in 32-bit and 64-bit programs) and the return value

Name	Return	EAX	RAX	Arguments
`read`	`???`	`3`	`0`	1: fd 2: ptr* 3: count
Attempts to read count bytes from the current position of the file into the buffer pointed to by ptr, the accumulator stores the true number of bytes read (if EOF less bytes would be read) and the position in the file is advanced by this many bytes. On error, -1 is returned instead				
`write`	`???`	`4`	`1`	1: fd 2: ptr* 3: count
Writes the buffer (count bytes pointed to by ptr) to the file				
`open`	`fd|error`	`5`	`2`	1: pathname* 2: flags 3: mode
Opens the file located at pathname using the given flags, on success returns the file descriptor (non-negative integer), on failure returns -1. The mode argument is for the permissions of the created file when O_CREAT is given as a flag and is otherwise ignored				
`close`	`status`	`6`	`3`	1: fd
Closes the file descriptor, returns 0 on success and -1 on failure				
`fstat`	`status`	`108`	`5`	1: fd 2: statbuf*
Writes information about the file descriptor into the `struct stat` pointed to by statbuf, returns 0 on success and -1 on failure				
`exit`	`???`	`1`	`60`	1: error
Exits the program with the specified error code				
`munmap`	`???`	`91`	`11`	1: addr 2: length
Frees the length bytes at the address addr, must exactly match its corresponding mmap call				
`mremap`	`?`	`?`	`?`	
???				
`mmap`	`???`	`90`	`9`	1: addr 2: length 3: prot 4: flags 5: fd 6: offset
Allocates length (must be a multiple of the page size [4096]) bytes, if addr is NULL (0) then any location is chosen else it will attempt to allocate at addr; prot is the protection flags for the memory, it is an bitwise OR of READ (1), WRITE (2), EXECUTE (4). Flags contains the mapping flags which is a bitwise OR of each flag such as MAP_PRIVATE (0x2, changes are private to the calling process and are not written back) and MAP_ANONYMOUS (0x20, allocates from main memory so not backed by a file as fd is ignored), fd can be set to -1 for anonymous mappings and offset is the offset from the start of the file (must be a multiple of the page size). eax/rax contains the ptr to the memory location allocated or an error between -4096 and -1. If a fd is given, the file is added to the programs virtual address space without being loaded into main memory (similar to swap memory). Additionally huge pages can be allocated with certain mappings and caution should be used when using this to access a file since race conditions can occur between processes if the file is not locked and the changes are not immediately written so use msync or fsync				

A fd (file descriptor) in Linux is an integer used to refer to a file, when opening a file the kernel returns an integer to the program to refer to the file with (lowest unused integer), and the kernel keeps track of the open file. Integers 0, 1

and 2 refer to stdin, stdout and stderr respectively, any negative fd is invalid so to specify no file, -1 is usually used.

7 Appendix

All of the code used/written has been appended below, it includes programs that can run on Linux or baremetal

7.1 Bash scripts

These bash scripts were used to run the code.

64-bit Linux code (Links together files if given more than one argument)

```
#!/bin/bash

# Assembles the assembly files it is given into object files in the same dir
# and name as the file it was given. Links together all object files given (and
# the ones it assembled) into a binary file in the directory and name of the
# first argument. If given '--' as an argument, all arguments after it will be
# passed to the executeable.

if [ $# -eq 0 ]; then
    echo "Must have at least one argument"
    exit 1
fi

objects=()
args=()
is_arg=false

for arg in "${@}"; do
    if [ $is_arg = true ]; then
        args+=($arg)
    elif [[ $arg = "--" ]]; then
        if [[ ${#objects[@]} = 0 ]]; then
            echo "Must give at least one file before the program arguments"
            exit 1
        else
            is_arg=true
        fi
    else
        file="$(pwd)/$arg"

        case $file in
            *.asm | *.s | *.S)
                object="${file%.*}.o"
                objects+=($object)
                nasm -f elf64 $file -o $object || exit 1 ;;
            *.o | *.a | *.so)
                objects+=($file) ;;
            *)
                echo "'$file' does not end in a correct file extention"
                exit 1 ;;
        esac
    fi
done

output="$(pwd)/${1%.*}.bin"
ld "${objects[@]}" -o $output || exit 1
$output "${args[@]}"
```

32-bit Linux code

```
#!/bin/bash

if [ $# -ne 1 ]; then
  echo "Incorrect Args"
  exit 1
fi

ASM="$(pwd)/$1"
if [[ $ASM == *.asm ]]; then
  BASE="${ASM%.asm}"
else
  echo "File does not end in .asm"
```

```
  exit 1
fi

OBJECT="${BASE}.o"

nasm -f elf32 $ASM -o $OBJECT
ld -m elf_i386 $OBJECT -o $BASE
$BASE
```

7.2 Hello world (Linux)

This program writes "Hello World" to stdout then exits without an error.

64-bit code:

```
section .data
    hello db 'Hello world', 0x0A ; Hello world followed by a newline
    ; $ is current mem location, then subtracts start of hello to get length
    length equ $ - hello ; Length of hello in bytes

section .text
    global _start ; Make _start accessible to the linker

_start:
    mov rax, 1 ; set syscall to 1 (sys_write)
    mov rdi, rax ; set arg1 to 1 (stdout)
    mov rsi, hello ; set arg2 to point to hello
    mov rdx, length ; set arg3 to length of hello
    syscall ; perform the syscall

    mov rax, 60 ; set syscall to 60 (exit)
    xor rdi, rdi ; set arg1 to 0 (success)
    syscall ; perform the syscall
```

32-bit code:

```
section .data
    hello db 'Hello world', 0x0A
    length equ $ - hello

section .text
    global _start

_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, hello
    mov edx, length
    int 80h

    mov eax, ebx
    xor ebx, ebx
    int 80h
```

7.3 Difference (Linux)

This program reads two inputs from stdin and writes their difference to stdout

```
section .bss
    print_buffer_len equ 128
    print_buffer resb print_buffer_len ; Reserve bytes of uninitilised memory

section .data
    prompt_text db 'Enter two values to find their difference', 0x0A
    prompt_len equ $ - prompt_text

section .text
    global _start

_start:
    ; print prompt text
    mov rax, 1
    mov rdi, rax
    mov rsi, prompt_text
```

```
    mov rdx, prompt_len
    syscall

    call read_rax
    mov rbx, rax
    call read_rax
    cmp rax, rbx
    jae skip_exchange
    xchg rax, rbx
skip_exchange:
    sub rax, rbx

    call print_rax
exit:
    xor rdi, rdi
    mov rax, 60
    syscall

; returns rax (value read)
read_rax:
    push rdi
    push rsi
    push rbx
    push rcx
    push rdx

    xor rax, rax
    xor rdi, rdi
    mov rsi, print_buffer
    mov rdx, print_buffer_len
    syscall

    ; Exit if returned value is greater than the buffer length
    cmp rax, print_buffer_len
    ja exit

    mov rcx, rax
    xor rax, rax
    mov rdi, 10
    xor rbx, rbx
read_loop:
    ; Exit if reached end of buffer and no newline
    dec rcx
    jc exit

    mov byte bl, [rsi]
    cmp rbx, 0x0A
    je read_return ; return if newline

    sub rbx, 48
    cmp rbx, 9
    ja exit ; exit if above 9 (so rbx is between 0 and 9 inclusive)

    mul rdi
    add rax, rbx
    inc rsi
    jmp read_loop

read_return:
    pop rdx
    pop rcx
    pop rbx
    pop rsi
    pop rdi
    ret

print_rax:
    push rdi
    push rsi
    push rdx
    push rbx
    push rax

    mov rsi, print_buffer
    xor rdi, rdi
    mov rbx, 10
```

```
print_loop_right:
    inc rdi
    xor rdx, rdx
    div rbx
    cmp rax, 0
    jne print_loop_right
    ; rsi now points to the location where the newline character should be
    add rsi, rdi
    mov byte [rsi], 0x0A

    ; read rax again
    mov rax, [rsp]
    inc rdi ; rdi now stores len of string
print_loop_left:
    dec rsi
    xor rdx, rdx
    div rbx
    add rdx, 48
    mov byte [rsi], dl
    cmp rax, 0
    jne print_loop_left

    mov rdx, rdi
    mov rax, 1
    mov rdi, rax
    syscall

    pop rax
    pop rbx
    pop rdx
    pop rsi
    pop rdi

    ret
```

7.4 Cat (Linux)

This program performs the cat command (it takes in one argument which is a filename and prints its contents). It uses the second arg (the first being the path) as the filename for the open syscall, it then calls the fstat syscall on the returned file descriptor. It gets the size of the file from fstat and allocates the file to memory (the size allocated is the next multiple of page size of the file size). It then writes the memory to stdout then deallocates the memory, closes the file and exits. It also writes to stderr and exits on critical errors (ones that would cause undefined behaviour) but ignores non-critical errors

```
; Cli args (stack)
; Syscalls: open, fstat, mmap, write, munmap, close, exit

section .bss
    stat_buffer resb stat_buffer_len ; size of the struct stat

section .data
    arg_error_text db 'Must be given a single argument', 0x0A
    arg_error_len equ $ - arg_error_text
    open_error_text db 'Failed to open file', 0x0A
    open_error_len equ $ - open_error_text
    fstat_error_text db 'Failed to call fstat', 0x0A
    fstat_error_len equ $ - fstat_error_text
    mmap_error_text db 'Failed to call mmap', 0x0A
    mmap_error_len equ $ - mmap_error_text
    stat_buffer_len equ 144

section .text
    global _start

_start:
    cmp qword [rsp], 2 ; [rsp] contains no. of args
    jne arg_exit
    mov rdi, [rsp+16] ; [rsp+8] contains ptr to path arg, [rsp+16] contains first arg

    ; open file, filename already in rdi, rdx is garbage as O_CREAT is not set
    xor rsi, rsi ; 0 for O_RDONLY (read only)
    mov rax, 2
    syscall
    cmp rax, 0
```

```
    jl open_exit ; exit if fd is negative (error)

    ; call fstat
    mov rdi, rax ; fd
    mov rsi, stat_buffer
    mov rax, 5
    syscall
    cmp rax, 0
    jne fstat_exit ; exit if an error is returned

    mov r8, rdi ; fd
    ; move size of file into rsi and round it up to a multiple of page size (4KiB)
    mov qword rsi, [stat_buffer+48]
    add rsi, 4096
    mov rax, 0xFFF ; Load mask
    not rax ; Not mask (all 1s but the 1s less than 4096)
    and rsi, rax
    mov r12, rsi ; store in r12 so its not overwritten

    ; Allocate memory contained by fd
    xor rdi, rdi ; NULL for any starting address
    mov rdx, 1 ; PROT_READ (prot)
    mov r10, 2 ; MAP_PRIVATE (flags)
    xor r9, r9 ; 0 offset
    mov rax, 9
    syscall
    cmp rax, 0
    jl mmap_exit ; exit if an error is returned

    mov rsi, rax ; ptr to address
    mov rdx, [stat_buffer+48]
    mov rdi, 1 ; stdout
    mov rax, rdi
    syscall ; print buffer

    ; munmap buffer (not required as we exit anyway)
    mov rdi, rsi
    mov rsi, r12
    mov rax, 11
    syscall ; Ignore potential error since we exit anyway

    ; close file (not required as we exit anyway)
    mov rdi, r8
    mov rax, 3 ; line could be removed, the fd is likely 3 as its the lowest unused fd
    syscall ; Ignore potential error since we exit anyway

    ; exit
    xor rdi, rdi
    mov rax, 60
    syscall

mmap_exit:
    mov rsi, mmap_error_text
    mov rdx, mmap_error_len
    jmp exit
fstat_exit:
    mov rsi, fstat_error_text
    mov rdx, fstat_error_len
    jmp exit
open_exit:
    mov rsi, open_error_text
    mov rdx, open_error_len
    jmp exit
arg_exit:
    mov rsi, arg_error_text
    mov rdx, arg_error_len
exit: ; rsi - ptr, rdx - len
    ; print to stderr
    mov rdi, 2
    mov rax, 1
    syscall
    ; exit
    mov rdi, 1
    mov rax, 60
    syscall
```

7.5 Hello world (Baremetal)

This program prints "Hello world" to the vga-mode buffer.

```
BITS 16                     ; Instruct NASM to produce 16 bit (real mode) code
org 0x7c00                  ; Set the origin to 0x7c00 which is where BIOS loads the bootloader

start:
    mov ax, 0xb800          ; Video memory segment (0xb800)
    mov es, ax              ; Load ES with the video memory segment

    xor di, di              ; Start of memory segment (using di as an offset)
    mov cx, 2000            ; 80 by 25 chars

clearscreen:
    mov byte [es:di], 0x20  ; Write space to video memory at ES:DI
    inc di

    mov byte [es:di], 0x07  ; Write attribute (light gray on black) to video memory at ES:DI
    inc di

    dec cx
    jnz clearscreen         ; Repeat if cx is non-zero

    lea si, [message]       ; Load the address of 'message' into SI
    xor di, di              ; Start at the beginning of the video memory (offset 0)
    mov cx, message_len

print_line:
    mov byte al, [si]       ; Move byte from message into the video memory
    mov byte [es:di], al

    dec cx
    jz halt

    inc si
    add di, 2

    jmp print_line

halt:
    cli                     ; Disable interrupts
    hlt                     ; Halt (Only resumes on an interrupt however they were disabled)

message db 'Hello world'
message_len equ $ - message

; Boot sector padding and signature
times 510-($-$$) db 0       ; Pad the boot sector to 510 bytes (ensuring the total size is 512 bytes
    )
dw 0xAA55                   ; Boot sector signature (0xAA55), required for a valid bootable sector
```